Open in Colab

빅데이터분석프로그래밍

파이썬 설치 확인

In [1]:
```
import site
site.getsitepackages()
```

Out[1]:
```
['/usr/local/lib/python3.12/dist-packages',
 '/usr/lib/python3/dist-packages',
 '/usr/lib/python3.12/dist-packages']
```

In [2]:
```
import sys
import pandas as pd
import numpy as np
import seaborn as sns

print(sys.version)
print(pd.__version__)
print(np.__version__)
print(sns.__version__)
```

```
3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
2.2.2
2.0.2
0.13.2
```

코랩에서 한글 처리

In [3]:
```
!pip install koreanize_matplotlib
```

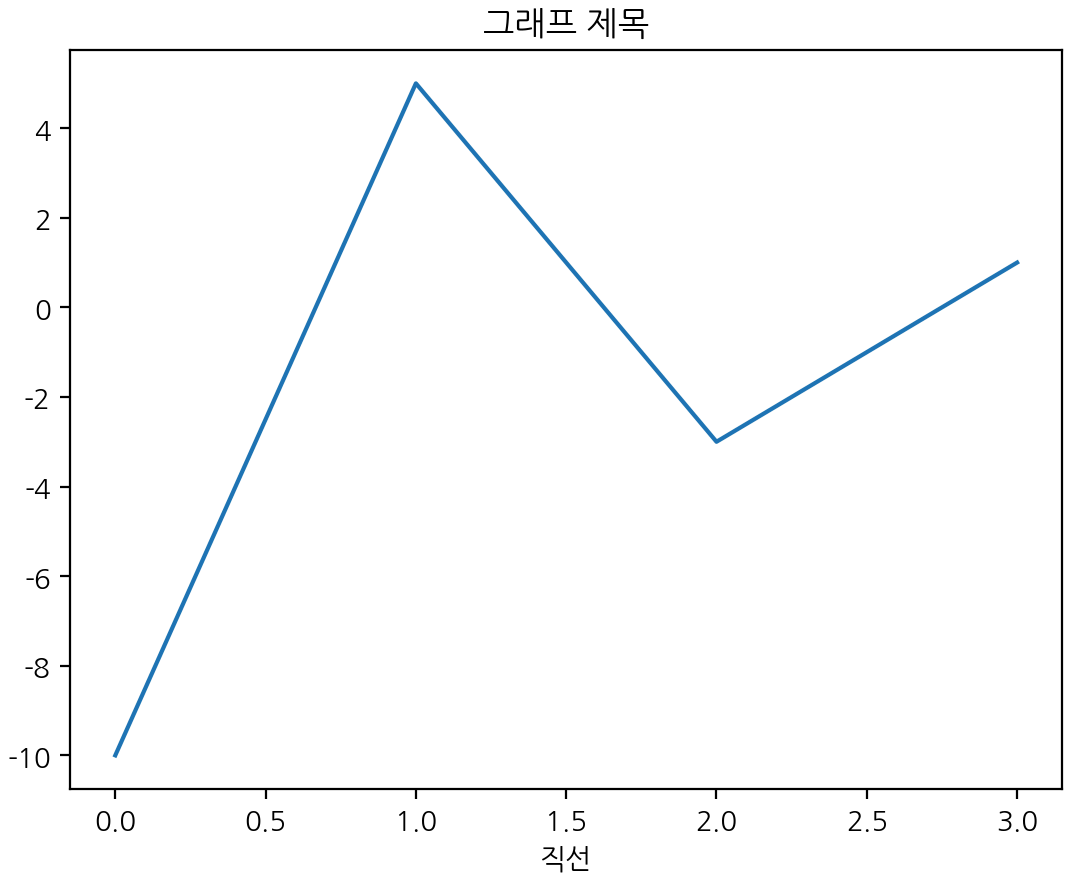
```
Collecting koreanize_matplotlib
  Downloading koreanize_matplotlib-0.1.1-py3-none-any.whl.metadata (992 by
tes)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dis
t-packages (from koreanize_matplotlib) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.
12/dist-packages (from matplotlib->koreanize_matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib->koreanize_matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python
3.12/dist-packages (from matplotlib->koreanize_matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python
3.12/dist-packages (from matplotlib->koreanize_matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/di
st-packages (from matplotlib->koreanize_matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.1
2/dist-packages (from matplotlib->koreanize_matplotlib) (26.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist
-packages (from matplotlib->koreanize_matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.
12/dist-packages (from matplotlib->koreanize_matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/pyth
on3.12/dist-packages (from matplotlib->koreanize_matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib->koreanize_matplotlib) (1.
17.0)
Downloading koreanize_matplotlib-0.1.1-py3-none-any.whl (7.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/7.9 MB ? eta -:--:--
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.8/7.9 MB 143.4 MB/s eta 0:0
0:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.9/7.9 MB 156.4 MB/s eta 0:0
0:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.9/7.9 MB 84.9 MB/s eta 0:00:
00
Installing collected packages: koreanize_matplotlib
Successfully installed koreanize_matplotlib-0.1.1
```

In [4]:
```
# 그림 선명하게
%config InlineBackend.figure_format = 'retina'
```

In [5]:
```
# 데이터프레임의 소수점 이하 n(2)자리 출력 설정
pd.set_option('display.precision', 2)
```

In [6]:
```
import matplotlib.pyplot as plt
import koreanize_matplotlib

plt.plot([-10, 5, -3, 1])
plt.title('그래프 제목')
plt.xlabel('직선')
plt.show()
```



수업에서 사용할 데이터를 다음 깃에서 복사

- https://github.com/ai7dnn/data.git

> 수업에서 파일 사용 시
>
> - 현재 하부 폴더: data
> - 파일: data/파일이름

In [7]:
```
!git clone https://github.com/ai7dnn/data.git
```

```
Cloning into 'data'...
remote: Enumerating objects: 50, done.
remote: Counting objects: 100% (50/50), done.
remote: Compressing objects: 100% (48/48), done.
remote: Total 50 (delta 12), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (50/50), 5.71 MiB | 5.38 MiB/s, done.
Resolving deltas: 100% (12/12), done.
```

In [8]:
```
import pandas as pd

exam = pd.read_csv('data/exam.csv')
exam.head()
```

Out[8]:

	id	nclass	math	english	science
0	1	1	50	98	50
1	2	1	60	97	60
2	3	1	45	86	78
3	4	1	30	98	58
4	5	2	25	80	65

In [9]:
```
exam.agg("mean")
```

Out[9]:

	0
id	10.50
nclass	3.00
math	57.45
english	84.90
science	59.45

dtype: float64

In [10]:
```
exam.agg(["mean", "max", "min", "median"])
```

Out[10]:

	id	nclass	math	english	science
mean	10.5	3.0	57.45	84.9	59.45
max	20.0	5.0	90.00	98.0	98.00
min	1.0	1.0	20.00	56.0	12.00
median	10.5	3.0	54.00	86.5	62.50

In [11]:
```
exam.mean()
```

Out[11]:

	0
id	10.50
nclass	3.00
math	57.45
english	84.90
science	59.45

dtype: float64

In [12]:
```
exam.agg(mean_math = ('math', 'mean'))
```

Out[12]:

	math
mean_math	57.45

In [13]:
```
import numpy as np
df = pd.DataFrame([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9],
                   [np.nan, np.nan, 10]],
                   columns=['A', 'B', 'C'])
df
```

Out[13]:

	A	B	C
0	1.0	2.0	3
1	4.0	5.0	6
2	7.0	8.0	9
3	NaN	NaN	10

In [14]:
```
df.agg(['sum', 'mean', 'median', 'min', 'max', 'count'])
```

Out[14]:

	A	B	C
sum	12.0	15.0	28.0
mean	4.0	5.0	7.0
median	4.0	5.0	7.5
min	1.0	2.0	3.0
max	7.0	8.0	10.0
count	3.0	3.0	4.0

In [15]:
```
df.agg(['sum', 'mean', 'median', 'min', 'max', 'count'], axis=0)
```

Out[15]:

	A	B	C
sum	12.0	15.0	28.0
mean	4.0	5.0	7.0
median	4.0	5.0	7.5
min	1.0	2.0	3.0
max	7.0	8.0	10.0
count	3.0	3.0	4.0

In [16]:
```
df.agg(['sum', 'mean', 'median', 'min', 'max', 'count'], axis='index')
```

Out[16]:

	A	B	C
sum	12.0	15.0	28.0
mean	4.0	5.0	7.0
median	4.0	5.0	7.5
min	1.0	2.0	3.0
max	7.0	8.0	10.0
count	3.0	3.0	4.0

In [17]:
```
df.agg(['sum', 'mean', 'median', 'min', 'max', 'count'], axis=1)
```

Out[17]:

	sum	mean	median	min	max	count
0	6.0	2.0	2.0	1.0	3.0	3.0
1	15.0	5.0	5.0	4.0	6.0	3.0
2	24.0	8.0	8.0	7.0	9.0	3.0
3	10.0	10.0	10.0	10.0	10.0	1.0

In [18]:
```
df.agg(['sum', 'mean', 'median', 'min', 'max', 'count'], axis='columns')
```

Out[18]:

	sum	mean	median	min	max	count
0	6.0	2.0	2.0	1.0	3.0	3.0
1	15.0	5.0	5.0	4.0	6.0	3.0
2	24.0	8.0	8.0	7.0	9.0	3.0
3	10.0	10.0	10.0	10.0	10.0	1.0

집단별 요약 통계량 구하기

In [ ]:
```
exam.tail()
```

Out[ ]:

	id	nclass	math	english	science
15	16	4	58	98	65
16	17	5	65	68	98
17	18	5	80	78	90
18	19	5	89	68	87
19	20	5	78	83	58

In [ ]:
```
exam.groupby('nclass').agg(mean_math = ('math', 'mean'))
```

Out[ ]:

nclass	mean_math
1	46.25
2	61.25
3	45.00
4	56.75
5	78.00

In [ ]:
```
exam.groupby('nclass').agg(max_math = ('math', 'max'))
```

Out[ ]:

nclass	max_math
1	60
2	90
3	65
4	75
5	89

In [ ]:
```
exam.groupby('nclass').mean()
```

Out[ ]:

nclass	id	math	english	science
1	2.5	46.25	94.75	61.50
2	6.5	61.25	84.25	58.25
3	10.5	45.00	86.50	39.25
4	14.5	56.75	84.75	55.00
5	18.5	78.00	74.25	83.25

In [ ]:
```
exam.groupby('nclass').mean()[["math"]]
```

Out[ ]:

nclass	math
1	46.25
2	61.25
3	45.00
4	56.75
5	78.00

In [ ]:
```
exam.groupby('nclass').mean()[["math", "english", "science"]]
```

Out[ ]:

nclass	math	english	science
1	46.25	94.75	61.50
2	61.25	84.25	58.25
3	45.00	86.50	39.25
4	56.75	84.75	55.00
5	78.00	74.25	83.25

In [ ]:
```
exam.groupby('nclass').min()
```

Out[ ]:

nclass	id	math	english	science
1	1	30	86	50
2	5	25	78	25
3	9	20	65	15
4	13	46	56	12
5	17	65	68	58

In [ ]:
```
exam.groupby('nclass').agg("mean")
```

Out[ ]:

nclass	id	math	english	science
1	2.5	46.25	94.75	61.50
2	6.5	61.25	84.25	58.25
3	10.5	45.00	86.50	39.25
4	14.5	56.75	84.75	55.00
5	18.5	78.00	74.25	83.25

In [ ]:
```
exam.groupby('nclass').agg(["mean", "median"])
```

Out[ ]:

	id		math		english		science	
	mean	median	mean	median	mean	median	mean	median
nclass								
1	2.5	2.5	46.25	47.5	94.75	97.5	61.50	59.0
2	6.5	6.5	61.25	65.0	84.25	84.5	58.25	55.0
3	10.5	10.5	45.00	47.5	86.50	91.5	39.25	38.5
4	14.5	14.5	56.75	53.0	84.75	92.5	55.00	65.0
5	18.5	18.5	78.00	79.0	74.25	73.0	83.25	88.5

In [ ]:

```
# 인자 사용 방법: 열 이름은 키워드인 반면, 값 ('열명', '함수명')은 열에서 값을 계산하는
# 새_열이름 = ('열명', '함수명')
exam.groupby('nclass').agg(meanMath = ('math', 'mean'))
```

Out[ ]:

	meanMath
nclass	
1	46.25
2	61.25
3	45.00
4	56.75
5	78.00

In [ ]:

```
exam.groupby('nclass').agg(meanMath = ('math', 'mean'), meanEng = ('engli
```

Out[ ]:

	meanMath	meanEng
nclass		
1	46.25	94.75
2	61.25	84.25
3	45.00	86.50
4	56.75	84.75
5	78.00	74.25

In [ ]:

```
# 인자 사용 방법: 사전 형식
# {'열명': '함수명'}
exam.groupby('nclass').agg({'math': 'mean'})
```

Out[ ]:

nclass	math
1	46.25
2	61.25
3	45.00
4	56.75
5	78.00

In [ ]:
```
# dict of axis labels -> functions, function names or list of such.
# {'열명': ['함수명1', '함수명2', ...]}
exam.groupby('nclass').agg({'math': ['mean', 'min', 'max']})
```

Out[ ]:

	math		
nclass	mean	min	max
1	46.25	30	60
2	61.25	25	90
3	45.00	20	65
4	56.75	46	75
5	78.00	65	89

(알아 두면 좋아요) 변수를 인덱스로 바꾸지 않기

In [ ]:
```
exam.groupby('nclass').agg(mean_math = ('math', 'mean'))
```

Out[ ]:

nclass	mean_math
1	46.25
2	61.25
3	45.00
4	56.75
5	78.00

In [ ]:
```
exam.groupby('nclass', as_index=False).agg(mean_math = ('math', 'mean'))
```

Out[ ]:

	nclass	mean_math
0	1	46.25
1	2	61.25
2	3	45.00
3	4	56.75
4	5	78.00

DataFrameGroupBy 객체

In [ ]:
```
exam.groupby('nclass')
```

Out[ ]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7d9193e7cac0>
```

In [ ]:
```
type(exam.groupby('nclass'))
```

Out[ ]:

pandas.core.groupby.generic.DataFrameGroupBy

def __init__(obj: NDFrameT, keys: _KeysArgType | None=None, axis: int=0, l
evel: IndexLabel | None=None, grouper: ops.BaseGrouper | None=None, ex
clusions: frozenset[Hashable] | None=None, selection: IndexLabel | None=
None, as_index: bool=True, sort: bool=True, group_keys: bool | lib.NoDefaul
t=True, squeeze: bool=False, observed: bool=False, mutated: bool=False, d
ropna: bool=True) -> None

Class for grouping and aggregating relational data.

See aggregate, transform, and apply functions on this object.

It's easiest to use obj.groupby(...) to use GroupBy, but you can also do:

In [ ]:
```
range(10)
```

Out[ ]:
```
range(0, 10)
```

In [ ]:
```
list(range(10))
```

Out[ ]:
```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [ ]:
```
exam_gb = exam.groupby('nclass')
exam_gb
```

Out[ ]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7d919aa99090>
```

In [ ]:
```
list(exam_gb)
```

Out[ ]:
```
[(1,
    id  nclass  math  english  science
 0   1       1    50       98       50
 1   2       1    60       97       60
 2   3       1    45       86       78
 3   4       1    30       98       58),
(2,
    id  nclass  math  english  science
 4   5       2    25       80       65
 5   6       2    50       89       98
 6   7       2    80       90       45
 7   8       2    90       78       25),
(3,
     id  nclass  math  english  science
 8    9       3    20       98       15
 9   10       3    50       98       45
 10  11       3    65       65       65
 11  12       3    45       85       32),
(4,
     id  nclass  math  english  science
 12  13       4    46       98       65
 13  14       4    48       87       12
 14  15       4    75       56       78
 15  16       4    58       98       65),
(5,
     id  nclass  math  english  science
 16  17       5    65       68       98
 17  18       5    80       78       90
 18  19       5    89       68       87
 19  20       5    78       83       58)]
```

In [ ]:
```
for key, value in exam_gb:
    print(key, '\n')
    print(type(value))
    print(value, '\n', '-'*40, '\n')
```

```
1

<class 'pandas.core.frame.DataFrame'>
   id  nclass  math  english  science
0   1       1    50       98       50
1   2       1    60       97       60
2   3       1    45       86       78
3   4       1    30       98       58
 ----------------------------------------

2

<class 'pandas.core.frame.DataFrame'>
   id  nclass  math  english  science
4   5       2    25       80       65
5   6       2    50       89       98
6   7       2    80       90       45
7   8       2    90       78       25
 ----------------------------------------

3

<class 'pandas.core.frame.DataFrame'>
    id  nclass  math  english  science
8    9       3    20       98       15
9   10       3    50       98       45
10  11       3    65       65       65
11  12       3    45       85       32
 ----------------------------------------

4

<class 'pandas.core.frame.DataFrame'>
    id  nclass  math  english  science
12  13       4    46       98       65
13  14       4    48       87       12
14  15       4    75       56       78
15  16       4    58       98       65
 ----------------------------------------

5

<class 'pandas.core.frame.DataFrame'>
    id  nclass  math  english  science
16  17       5    65       68       98
17  18       5    80       78       90
18  19       5    89       68       87
19  20       5    78       83       58
 ----------------------------------------
```

여러 요약 통계량 한 번에 구하기

In [ ]:
```
# nclass별로 분리
# 수학 점수 평균
# 수학 점수 합계
# 수학 점수 중앙값
# 빈도(학생 수)
```

In [ ]:
```
exam.groupby('nclass').agg(mean_math   = ('math', 'mean'),
                           sum_math    = ('math', 'sum'),
                           median_math = ('math', 'median'),
                           count       = ('nclass', 'count'))
```

Out[ ]:

nclass	mean_math	sum_math	median_math	count
1	46.25	185	47.5	4
2	61.25	245	65.0	4
3	45.00	180	47.5	4
4	56.75	227	53.0	4
5	78.00	312	79.0	4

(알아 두면 좋아요) 모든 변수의 요약 통계량 한 번에 구하기

In [ ]:
```
exam.groupby('nclass').mean()
```

Out[ ]:

nclass	id	math	english	science
1	2.5	46.25	94.75	61.50
2	6.5	61.25	84.25	58.25
3	10.5	45.00	86.50	39.25
4	14.5	56.75	84.75	55.00
5	18.5	78.00	74.25	83.25

In [ ]:
```
exam.groupby('nclass').agg(['mean', 'median'])
```

Out[ ]:

	id		math		english		science	
	mean	median	mean	median	mean	median	mean	median
nclass								
1	2.5	2.5	46.25	47.5	94.75	97.5	61.50	59.0
2	6.5	6.5	61.25	65.0	84.25	84.5	58.25	55.0
3	10.5	10.5	45.00	47.5	86.50	91.5	39.25	38.5
4	14.5	14.5	56.75	53.0	84.75	92.5	55.00	65.0
5	18.5	18.5	78.00	79.0	74.25	73.0	83.25	88.5

집단별로 다시 집단 나누기

In [ ]:
```
import pandas as pd
```

```
mpg = pd.read_csv('data/mpg.csv')
mpg.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 234 entries, 0 to 233
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   manufacturer  234 non-null    object
 1   model         234 non-null    object
 2   displ         234 non-null    float64
 3   year          234 non-null    int64
 4   cyl           234 non-null    int64
 5   trans         234 non-null    object
 6   drv           234 non-null    object
 7   cty           234 non-null    int64
 8   hwy           234 non-null    int64
 9   fl            234 non-null    object
 10  category      234 non-null    object
dtypes: float64(1), int64(4), object(6)
memory usage: 20.2+ KB
```

In [ ]:
```
mpg.columns
```

Out[ ]:
```
Index(['manufacturer', 'model', 'displ', 'year', 'cyl', 'trans', 'drv',
'cty',
       'hwy', 'fl', 'category'],
      dtype='object')
```

In [ ]:
```
mpg.sample(frac=.02)
```

Out[ ]:

	manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl
84	ford	f150 pickup 4wd	4.2	1999	6	manual(m5)	4	14	17	r
77	ford	explorer 4wd	4.0	1999	6	auto(l5)	4	14	17	r
118	hyundai	tiburon	2.0	2008	4	auto(l4)	f	20	27	r
34	chevrolet	malibu	3.1	1999	6	auto(l4)	f	18	26	r
88	ford	f150 pickup 4wd	5.4	1999	8	auto(l4)	4	11	15	r

In [ ]:
```
# 제조 회사 및 구동 방식별 분리
# cty 평균 구하기
mpg.groupby(['manufacturer', 'drv']).agg(mean_cty = ('cty', 'mean')) # 시
```

Out[ ]:

manufacturer	drv	mean_cty
audi	4	16.82
	f	18.86
chevrolet	4	12.50
	f	18.80
	r	14.10
dodge	4	12.00
	f	15.82
ford	4	13.31
	r	14.75
honda	f	24.44
hyundai	f	18.64
jeep	4	13.50
land rover	4	11.50
lincoln	r	11.33
mercury	4	13.25
nissan	4	13.75
	f	20.00
pontiac	f	17.00
subaru	4	19.29
toyota	4	14.93
	f	21.37
volkswagen	f	20.93

In [ ]:

```
mpg.groupby(['drv', 'manufacturer']).agg(mean_cty = ('cty', 'mean')) # 시
```

Out[ ]:

drv	manufacturer	mean_cty
4	audi	16.82
	chevrolet	12.50
	dodge	12.00
	ford	13.31
	jeep	13.50
	land rover	11.50
	mercury	13.25
	nissan	13.75
	subaru	19.29
	toyota	14.93
f	audi	18.86
	chevrolet	18.80
	dodge	15.82
	honda	24.44
	hyundai	18.64
	nissan	20.00
	pontiac	17.00
	toyota	21.37
	volkswagen	20.93
r	chevrolet	14.10
	ford	14.75
	lincoln	11.33

In [ ]:

```
# 제조 회사와 차종별 분리
# cty 평균 구하기
# hwy 평균 구하기
mpg.groupby(['manufacturer', 'category']).agg(mean_cty = ('cty', 'mean'),
```

Out[ ]:

manufacturer	category	mean_cty	mean_hwy
audi	**compact**	17.93	26.93
	midsize	16.00	24.00
chevrolet	**2seater**	15.40	24.80
	midsize	18.80	27.60
	suv	12.67	17.11
dodge	**minivan**	15.82	22.36
	pickup	12.05	16.11
	suv	11.86	16.00
ford	**pickup**	13.00	16.43
	subcompact	15.89	23.22
	suv	12.89	17.78
honda	**subcompact**	24.44	32.56
hyundai	**midsize**	19.00	27.71
	subcompact	18.29	26.00
jeep	**suv**	13.50	17.62
land rover	**suv**	11.50	16.50
lincoln	**suv**	11.33	17.00
mercury	**suv**	13.25	18.00
nissan	**compact**	20.00	28.00
	midsize	20.00	27.43
	suv	13.75	18.00
pontiac	**midsize**	17.00	26.40
subaru	**compact**	19.75	26.00
	subcompact	19.50	26.00
	suv	18.83	25.00
toyota	**compact**	22.25	30.58
	midsize	19.86	28.29
	pickup	15.57	19.43
	suv	14.38	18.25
volkswagen	**compact**	20.79	28.50
	midsize	18.57	27.57
	subcompact	24.00	32.83

In [ ]:

```
## audi의 drv별 빈도
```

```
# audi 추출
# drv별 분리
# 빈도 구하기
mpg.query('manufacturer == "audi"').groupby(['drv']).agg(n = ('drv', 'cou
```

Out[ ]:

drv	n
4	11
f	7

In [ ]:

```
## chevrolet의 drv별 빈도

# chevrolet 추출
# drv별 분리
# 빈도 구하기
mpg.query('manufacturer == "chevrolet"') \
   .groupby(['drv']).agg(n = ('drv', 'count'))
```

Out[ ]:

drv	n
4	4
f	5
r	10

In [ ]:

```
mpg.query('manufacturer == "chevrolet"').groupby(['drv']).count()
```

Out[ ]:

drv	manufacturer	model	displ	year	cyl	trans	cty	hwy	fl	category
4	4	4	4	4	4	4	4	4	4	4
f	5	5	5	5	5	5	5	5	5	5
r	10	10	10	10	10	10	10	10	10	10

In [ ]:

```
mpg.query('manufacturer == "chevrolet"').groupby(['drv']).count()[['model
```

Out[ ]:

drv	model
4	4
f	5
r	10

In [ ]:

```
mpg.query('manufacturer == "chevrolet"').groupby(['drv']).count()[['model
```

Out[ ]:

drv	n
4	4
f	5
r	10

(알아 두면 좋아요) `value_counts()` 로 집단별 빈도 간단하게 구하기

In [ ]:
```
mpg.groupby('drv').agg(n = ('drv', 'count'))
```

Out[ ]:

drv	n
4	103
f	106
r	25

In [ ]:
```
mpg[['drv', 'cyl']].value_counts()
```

Out[ ]:
```
drv  cyl
f    4      58
4    8      48
f    6      43
4    6      32
     4      23
r    8      21
f    5       4
r    6       4
f    8       1
dtype: int64
```

In [ ]:
```
mpg[['drv', 'cyl']].value_counts().to_frame()
```

Out[ ]:

drv	cyl	0
f	4	58
4	8	48
f	6	43
4	6	32
	4	23
r	8	21
f	5	4
r	6	4
f	8	1

In [ ]:
```
mpg['drv'].value_counts().query('n > 100') # 오류 발생
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-23-f1595edfffaf> in <cell line: 1>()
----> 1 mpg['drv'].value_counts().query('n > 100') # 오류 발생

/usr/local/lib/python3.10/dist-packages/pandas/core/generic.py in __getattr__(self, name)
   5900         ):
   5901             return self[name]
-> 5902         return object.__getattribute__(self, name)
   5903
   5904     def __setattr__(self, name: str, value) -> None:

AttributeError: 'Series' object has no attribute 'query'
```

In [ ]:
```
mpg['drv'].value_counts().to_frame().query('drv > 100')
```

Out[ ]:

	drv
f	106
4	103

In [ ]:
```
# drv 빈도 구하기
# 데이터 프레임으로 바꾸기, 변수명 n으로 바꾸기
# axis 이름에 drv 지정
# n이 100을 초과한 경우 추출
mpg['drv'].value_counts() \
          .to_frame('n') \
          .rename_axis('drv') \
          .query('n > 100')
```

Out[ ]:

drv	n
f	106
4	103

In [ ]:
```
mpg['drv'].value_counts().to_frame('n') # 열 제목
```

Out[ ]:

	n
f	106
4	103
r	25

In [ ]:
```
mpg['drv'].value_counts().to_frame('n').rename_axis('drv') # index의 제목
```

Out[ ]:

drv	n
f	106
4	103
r	25

In [ ]:
```
mpg.groupby('drv').agg(n = ('drv', 'count'))
```

Out[ ]:

drv	n
4	103
f	106
r	25

In [ ]:
```
mpg.groupby('drv').agg(n = ('drv', 'count')).query('n > 100')
```

Out[ ]:

drv	n
4	103
f	106

`pandas` 함수 조합하기

In [ ]:
```
# suv 추출
# 합산 연비 변수 만들기
# 제조 회사별로 분리
```

```python
# 합산 연비 평균 구하기
# 내림차순 정렬
# 1~5위까지 출력, 시험 예상
mpg.query('category == "suv"') \
    .assign(total = (mpg['hwy'] + mpg['cty']) / 2) \
    .groupby('manufacturer') \
    .agg(mean_tot = ('total', 'mean')) \
    .sort_values('mean_tot', ascending = False) \
    .head(10)
```

Out[ ]:

manufacturer	mean_tot
subaru	21.92
toyota	16.31
nissan	15.88
mercury	15.62
jeep	15.56
ford	15.33
chevrolet	14.89
lincoln	14.17
land rover	14.00
dodge	13.93

In [ ]:

```python
mpg.query('category == "suv"') \
    .assign(total = (mpg['hwy'] + mpg['cty']) / 2) \
    .groupby('manufacturer') \
    .agg(mean_tot = ('total', 'mean')) \
    .sort_values('mean_tot', ascending = False)
```

Out[ ]:

manufacturer	mean_tot
subaru	21.92
toyota	16.31
nissan	15.88
mercury	15.62
jeep	15.56
ford	15.33
chevrolet	14.89
lincoln	14.17
land rover	14.00
dodge	13.93

혼자서 해보기

06-7 데이터 합치기

In [ ]:
```
# 중간고사 데이터 만들기
test1 = pd.DataFrame({'id'      : [1, 2, 3, 4, 5],
                      'midterm' : [60, 80, 70, 90, 85]})

# 기말고사 데이터 만들기
test2 = pd.DataFrame({'id'    : [1, 2, 3, 4, 5],
                      'final' : [70, 83, 65, 95, 80]})
```

In [ ]:
```
test1  # test1 출력
```

Out[ ]:

	id	midterm
0	1	60
1	2	80
2	3	70
3	4	90
4	5	85

In [ ]:
```
test2  # test2 출력
```

Out[ ]:

	id	final
0	1	70
1	2	83
2	3	65
3	4	95
4	5	80

In [ ]:
```
pd.merge(test1, test2)
```

Out[ ]:

	id	midterm	final
0	1	60	70
1	2	80	83
2	3	70	65
3	4	90	95
4	5	85	80

In [ ]:
```
pd.merge(test1, test2, on = 'id')
```

Out[ ]:

	id	midterm	final
0	1	60	70
1	2	80	83
2	3	70	65
3	4	90	95
4	5	85	80

In [ ]:
```
pd.merge(test1, test2, on = 'id', how = 'inner')
```

Out[ ]:

	id	midterm	final
0	1	60	70
1	2	80	83
2	3	70	65
3	4	90	95
4	5	85	80

In [ ]:
```
test1.merge(test2, on = 'id', how = 'inner')
```

Out[ ]:

	id	midterm	final
0	1	60	70
1	2	80	83
2	3	70	65
3	4	90	95
4	5	85	80

In [ ]:
```
test1
```

Out[ ]:

	id	midterm
0	1	60
1	2	80
2	3	70
3	4	90
4	5	85

In [ ]:
```
test2
```

Out[ ]:

	id	final
0	1	70
1	2	83
2	3	65
3	4	95
4	5	80

다른 데이터를 활용해 변수 추가하기

In [ ]:
```
name = pd.DataFrame({'nclass'  : [1, 2, 3, 4, 5],
                     'teacher' : ['kim', 'lee', 'park', 'choi', 'jung']})
name
```

Out[ ]:

	nclass	teacher
0	1	kim
1	2	lee
2	3	park
3	4	choi
4	5	jung

In [ ]:
```
exam.sample(5)
```

Out[ ]:

	id	nclass	math	english	science
19	20	5	78	83	58
12	13	4	46	98	65
17	18	5	80	78	90
15	16	4	58	98	65
0	1	1	50	98	50

In [ ]:
```
pd.merge(exam, name, on ='nclass', how = 'inner')
```

Out[ ]:

	id	nclass	math	english	science	teacher
0	1	1	50	98	50	kim
1	2	1	60	97	60	kim
2	3	1	45	86	78	kim
3	4	1	30	98	58	kim
4	5	2	25	80	65	lee
5	6	2	50	89	98	lee
6	7	2	80	90	45	lee
7	8	2	90	78	25	lee
8	9	3	20	98	15	park
9	10	3	50	98	45	park
10	11	3	65	65	65	park
11	12	3	45	85	32	park
12	13	4	46	98	65	choi
13	14	4	48	87	12	choi
14	15	4	75	56	78	choi
15	16	4	58	98	65	choi
16	17	5	65	68	98	jung
17	18	5	80	78	90	jung
18	19	5	89	68	87	jung
19	20	5	78	83	58	jung

세로로 합치기: concat()

In [ ]:

```
# 학생 1~5번 시험 데이터 만들기
group_a = pd.DataFrame({'id'   : [1, 2, 3, 4, 5],
                        'test' : [60, 80, 70, 90, 85]})

# 학생 6~10번 시험 데이터 만들기
group_b = pd.DataFrame({'id'   : [6, 7, 8, 9, 10],
                        'test' : [70, 83, 65, 95, 80]})
```

In [ ]:

```
group_a  # group_a 출력
```

Out[ ]:

	id	test
0	1	60
1	2	80
2	3	70
3	4	90
4	5	85

In [ ]:
```
group_b  # group_b 출력
```

Out[ ]:

	id	test
0	6	70
1	7	83
2	8	65
3	9	95
4	10	80

In [ ]:
```
pd.concat([group_a, group_b])
```

Out[ ]:

	id	test
0	1	60
1	2	80
2	3	70
3	4	90
4	5	85
0	6	70
1	7	83
2	8	65
3	9	95
4	10	80

In [ ]:
```
pd.concat([group_a, group_b]).reset_index() # 원 index가 열로 이동, 새로운 시
```

Out[ ]:

	index	id	test
0	0	1	60
1	1	2	80
2	2	3	70
3	3	4	90
4	4	5	85
5	0	6	70
6	1	7	83
7	2	8	65
8	3	9	95
9	4	10	80

In [ ]:

```
pd.concat([group_a, group_b]).reset_index(drop = True) # 원 index 제거, 새로
```

Out[ ]:

	id	test
0	1	60
1	2	80
2	3	70
3	4	90
4	5	85
5	6	70
6	7	83
7	8	65
8	9	95
9	10	80

merge() 시험 예상

In [ ]:

```
import pandas as pd

df1 = pd.DataFrame([[1, 'apple'], [2, 'banana']],
                   columns = ['id', 'fruits'])
df1
```

Out[ ]:

	id	fruits
0	1	apple
1	2	banana

In [ ]:
```
df2 = pd.DataFrame([[1, 3000], [5, 500]],
                   columns = ['id', 'price'])
df2
```

Out[ ]:

	id	price
0	1	3000
1	5	500

In [ ]:
```
pd.merge(df1, df2)
```

Out[ ]:

	id	fruits	price
0	1	apple	3000

In [ ]:
```
pd.merge(df1, df2, how = 'inner')
```

Out[ ]:

	id	fruits	price
0	1	apple	3000

In [ ]:
```
df1.merge(df2)
```

Out[ ]:

	id	fruits	price
0	1	apple	3000

In [ ]:
```
df1.merge(df2, how='left')
```

Out[ ]:

	id	fruits	price
0	1	apple	3000.0
1	2	banana	NaN

In [ ]:
```
df2
```

Out[ ]:

	id	price
0	1	3000
1	5	500

In [ ]:
```
df1.merge(df2, how='right')
```

Out[ ]:

	id	fruits	price
0	1	apple	3000
1	5	NaN	500

In [ ]:
```
df1.merge(df2, how='outer')
```

Out[ ]:

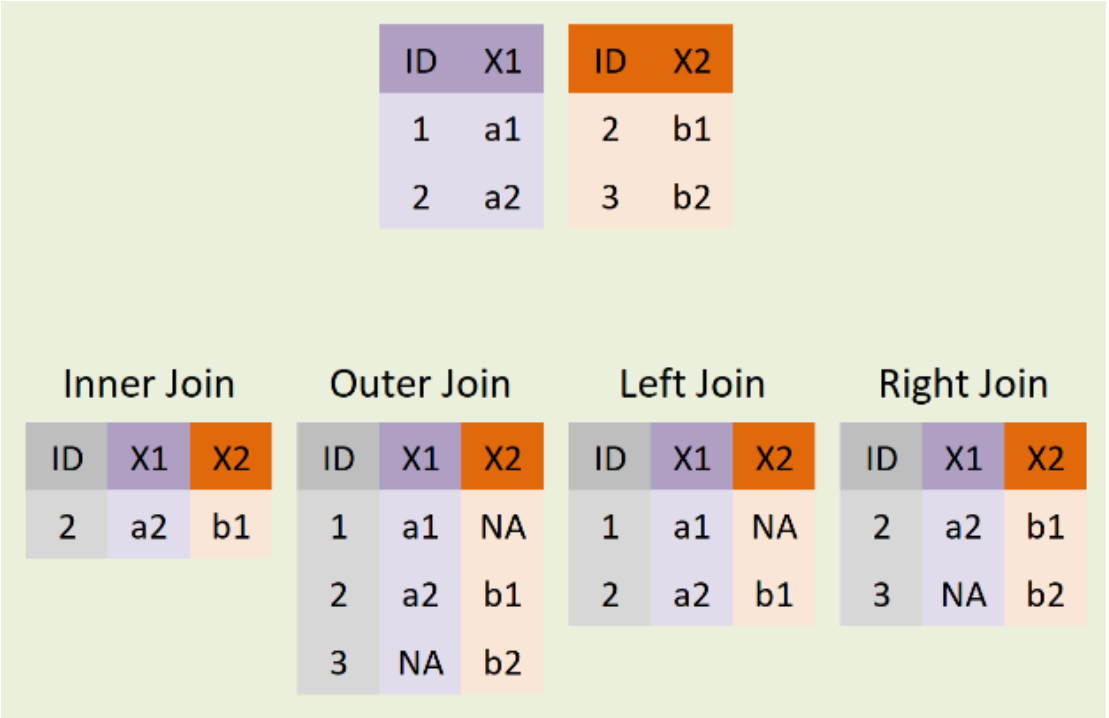
	id	fruits	price
0	1	apple	3000.0
1	2	banana	NaN
2	5	NaN	500.0

In [ ]:
```
df1.merge(df2, how='cross') # 두 프레임 모두에서 데카르트 곱을 생성하고 왼쪽 키의 순
```

Out[ ]:

	id_x	fruits	id_y	price
0	1	apple	1	3000
1	1	apple	5	500
2	2	banana	1	3000
3	2	banana	5	500

가로로 합치기 merge(): 행의 선택은?



정리하기

In [ ]:
```
# 추가한 변수를 pandas 코드에 바로 활용하기
exam.assign(total = exam['math'] + exam['english'] + exam['science']) \
    .sort_values('total').head()

## 6. 집단별로 요약하기
exam.groupby('nclass').agg(mean_math = ('math', 'mean'))

# 각 집단별로 다시 집단 나누기
mpg.groupby(['manufacturer', 'drv']).agg(mean_cty = ('cty', 'mean'))
```

```python
## 7. 데이터 합치기
pd.merge(test1, test2, how = 'left', on = 'id')  # 가로로 합치기
pd.concat([group_a, group_b])                    # 세로로 합치기
```

Out[ ]:

	id	test
0	1	60
1	2	80
2	3	70
3	4	90
4	5	85
0	6	70
1	7	83
2	8	65
3	9	95
4	10	80

종료